VTR UGE 2021- (CBCS)
School of Computing
B.Tech. – Computer Science and Engineering
Question Bank – Integrated Courses
Academic Year: Winter 2025 - 26

Course Category : Program Core

Course Code / Title : 10211CS226 / JAVA PROGRAMMING
Semester : Winter 25-26

Achievable Course Outcomes

CO1	Explain the fundamentals of Java and Object-Oriented Programming (OOP) concepts to solve problems.	K3
CO2	Apply the concepts of java inheritance and exception handling mechanisms for real world problem	K3
CO3	Solve the problems using Java Streams, Collections, UI Design and Threads	K3
CO4	Develop simple web applications using JSP, Servlet with JDBC and Hibernate Framework	K3
CO5	Build the web applications using Spring Framework	K3

Unit – I

Unit 1 Basics of OOP and JAVA **L-9Hours**

Unit Contents: Introduction to object-oriented programming - Features of Java - JVM- Keywords- Variables – Data types – Operators - Control statements -Command line arguments - Classes and Methods - Objects –Java Packages – Build-in packages - user-defined package -Access specifiers: private, protected, default, public - Constructors - Method Overloading - Type Casting - this keyword- Static – Arrays, Ulti-dimensional Arrays

Sl.No		**Two Marks Questions. (K1 Level Only)**	**Marks**	**Course Outcome**	**Level**
1.		Illustrate single inheritance using a simple class structure in Java.	2	CO1	K2
2.		Categorize the different types of operators used in Java.	2	CO1	K2
3.		Define an abstract class in Java and state why it cannot be instantiated.	2	CO1	K2
4.		Define an abstract method in Java and write its general syntax.	2	CO1	K2
5.		Summarize the key features of Java.	2	CO1	K2
6.		Differentiate local, instance, and static variables in Java.	2	CO1	K2

7.		List the types of polymorphism in Java and give one example for each.	2	CO1	K2
8.		Interpret the logic involved in converting temperature from Celsius to Fahrenheit.	2	CO1	K2
9.		Interpret the formula used to calculate the area of a rectangle.	2	CO1	K2
10.		Interpret the formula used to calculate the perimeter of a rectangle.	2	CO1	K2
11.		Classify the role of the Java Virtual Machine (JVM).	2	CO1	K2
12.		Define polymorphism in Java.	2	CO1	K2
13.		Point out the justification of the statement 'Java is platform independent'.	2	CO1	K2
14.		Distinguish between Classes and Objects.	2	CO1	K2
15.		Outline the syntax used for importing packages in a Java source file and illustrate it with an example.	2	CO1	K2

S. No		**Five Marks Questions. (K2 and above Level)**	**Marks**	**Course Outcome**	**Level**
1.		Apply the four fundamental OOP concepts in Java by developing suitable program examples that demonstrate each concept.	5	CO1	K3
2.		A computer science teacher assigned the students a programming task that calculates and displays grades based on test marks. The program should take a student's test mark as input and determine their grade using the following criteria: ● 90 to 100 → Grade A ● 80 to 89 → Grade B ● 70 to 79 → Grade C ● 60 to 69 → Grade D ● 45 to 59 → Grade E ● Below 45 → Grade F Develop and implement a Java program that accepts the test mark from the user and applies the given grading criteria to compute and display the appropriate grade.	5	CO1	K3
3.		Vikram, a system performance tester, often receives sets of numerical readings that represent processing times recorded during multiple test runs. To quickly identify the highest processing time, he wants to write a Java program that can take input directly from the command line. The task is to help Vikram by writing a java program that: a) Accepts a list of integers through command-line arguments. b) Converts all the arguments into integer values and stores them in an array.	5	CO1	K3

		c) Processes the array to find the maximum number among the inputs. d) Displays the highest value as the result. Demonstrate a Java program using command-line arguments to perform these steps and display the maximum value.			
4.		Implement different types of constructors (default, parameterized, and copy) in Java with suitable examples.	5	CO1	K3
5.		A student registration system must create student objects in three ways: • with no details (default values), • with name and department provided, • by copying details from an existing student object. Illustrate the use of default, parameterized, and copy-style constructors to create student objects in these three situations.	5	CO1	K3
6.		Geetha, a data analyst, frequently works with numerical datasets and often needs to perform quick calculations on the values she receives. To simplify her workflow, she wants to demo a Java program that can accept a set of numbers directly from the command line and compute their total. Your task is to help Geetha by writing a program that: 1. Accepts a series of integers through command-line arguments. 2. Converts the arguments into integer values and stores them in an array. 3. Calculates the sum of all the numbers. 4. Displays the final sum to the user. Demonstrating a Java program using command-line arguments to perform these steps and display the result.	5	CO1	K3
7.		Discuss on Variables. List the various types of variables with a simple program.	5	CO1	K3
8.		Preethi, a software developer, is working on a project where she needs to reverse the order of a list of numbers. This functionality is required to display a reversed sequence of numbers for a data processing tool she is developing. The numbers are provided as input in a sequence, and Preethi needs to reverse their order to meet the system's output requirements. Write a java program that takes a list of numbers as input and outputs them in reverse order.	5	CO1	K3
9.		Sam is demonstrating a java program to ensure data redundancy by copying the contents of one array into another. Creating a duplicate array makes sure that the original data remains preserved and can be reused for further operations or kept as a backup. Your task is to help Sam by writing a program that: 1. Accepts an integer N, representing the size of an array. 2. Reads N integers and stores them in an initial array, arr1. 3. Copies all elements from arr1 into a new array, arr2. 4. Displays all the elements of arr2.	5	CO1	K3

		Write a java program to perform these steps and display the duplicated array.			
10.		Explain Class, Method and Object? Show the syntax to define these in java with a simple program.	5	CO1	K3
11.		Discuss in detail the various operators in Java with suitable Java code examples.			
12.		Demonstrate the process that converts a long value to byte and back to double, and discuss the typecasting steps involved and their impact on data.	5	CO1	K3
13.		A calculator program needs to add: two integers, two floating numbers, and three integers. Demonstrate the use of method overloading to support all three addition operations and describe the process by which Java selects the correct method.	5	CO1	K3
14.		Summarize the characteristics of static fields and static methods in Java, highlighting their purpose and behavior within a class.	5	CO1	K3
15.		Demonstrate the use of arrays in finding a smallest number in a list of similar type.	5	CO1	K3

Unit – II

Unit 2 Inheritance, Exception Handling and string **L-9Hours**

Unit Contents: Basics of Inheritance - Forms of Inheritance - Super keyword – Final - Method Overriding –Abstraction: Abstract Classes - Interfaces. Exception Handling: Java Exception Hierarchy - Exception Types - Throwing and Catching exceptions -Declaring New Exception Types – custom exceptions. String class – Immutable String - String handling methods

S.No		**Two Mark Questions.**	**Marks**	**Course Outcome**	**Level**
1.		Distinguish between Abstract class and Interface	2	CO2	K2
2.		List the forms of inheritance supported in Java.	2	CO2	K2

3.		Give the output for the following program
		public class CommandArgs
		{
		public static void main(String [] args)
		{
		String s1 = args[1];
		String s2 = args[2];
		String s3 = args[3];
		System.out.print(" args[2] = " + s2);
		}
		}
			2	CO2	K2

4.		Exemplify method overriding in Java with one rule.	2	CO2	K2
5.		A banking application defines an interest rate that should not be changed once assigned. Show the Java keyword used to prevent modification of this value and mention its purpose.	2	CO2	K2

6.		Give the output for the following program
		try
		{
		int x = 0;
		int y = 5 / x;
		}
		catch (Exception e)
		{
		System.out.println("Exception");
		}
		catch (ArithmeticException ae)
		{
		System.out.println(" Arithmetic Exception");
		}
		System.out.println("finished");
			2	CO2	K2

7.		Summarize the use of super keyword in Java.	2	CO2	K2
8.		List out the types of inheritance in java.	2	CO2	K2
9.		Can you finally block get executed if either try or catch blocks are returning the control? Justify.	2	CO2	K2
10.		Discuss how method overriding works in Java and infer its impact on runtime behavior.	2	CO2	K2
11.		Interpret the difference between checked and unchecked exceptions in Java.	2	CO2	K2
12.		Classify the common methods provided by the String class and illustrate their conceptual usage.	2	CO2	K2
13.		Compare how does appending operations differ between String and StringBuffer in Java.	2	CO2	K2
14.		Differentiate String and String Buffer classes.	2	CO2	K2
15.		What is immutable string in java.	2	CO2	K2

S. No		**Five Marks Questions**	**Marks**	**Course Outcome**	**Level**
1.		A smart classroom system needs to support both teaching functionalities and attendance tracking functionalities. Since a single class must acquire behaviors from two different sources, construct a Java program to demonstrate multiple inheritance using interfaces in Java.	5	CO2	K3
2.		Implement Java programs to demonstrate the use of the following exception handling mechanisms: Try/Catch, Throw, Finally, and Throws.	5	CO2	K3
3.		Illustrate the following String handling methods with a Java program. a) length, b) substring, c) charAt, d) equals, e) toUpperCase	5	CO2	K3
4.		Infer the use of throw and throws keyword in exception handling.	5	CO2	K3
5.		Apply the concept of multiple inheritance in object-oriented programming by providing suitable examples.	5	CO2	K3
6.		Outline the use of super keyword to invoke a superclass's variable, constructor and methods. Illustrate with example program.	5	CO2	K2

7.		Build a menu-driven calculator program that performs four operations. Use try–catch blocks to handle invalid inputs, division by zero, and other arithmetic exceptions. Provide program code with sample executions.	5	CO2	K3
8.		Explain the following terms with respect to exception handling: a. Try/Catch b. Throw c. Finally d. Throws	5	CO2	K2
9.		Apply the concept of exception handling in Java by writing a program that demonstrates a try block likely to generate three types of exceptions. Incorporate necessary catch blocks to handle each exception appropriately.	5	CO2	K3
10.		Apply the concepts of the String class in Java to design a program that reads a sentence from the user and performs the following operations using built-in String methods: 1. Count the number of vowels 2. Reverse the string 3. Convert the entire string to uppercase 4. Replace all spaces with hyphens 5. Check whether the string is a palindrome. Write the Java program, explain the working of each method used, and illustrate the output for different sample inputs.	5	CO2	K3

Unit – III

Unit 3 Collection and Problem Solving

Unit Contents: Java.lang.Object – Array of Objects – java.lang.Arrays – Collection: ArrayList, LinkedList, Vector, HashMap, TreeMap - Debugging and Bug Fixing – Logic Mistake – Infinite Loops – Timeout error. Thread - Life Cycle - Multi-Threaded Programming — Thread Synchronization.

S. No		**Two Marks Questions.**	**Marks**	**Course Outcome**	**Level**
1.		Discuss the use of an LinkedList in Java with a simple example.	2	CO3	K2
2.		Differentiate Vector and ArrayList in Java Collections.	2	CO3	K2
3.		Outline the purpose of the Vector class?	2	CO3	K2
4.		Distinguish between HashMap and TreeMap.	2	CO3	K2
5.		Classify the types of Map in Java.	2	CO3	K2

6.		Give the output for the following program
		public class BoolTest
		{
		public static void main(String [] args)
		{
		int result = 0;
		Boolean b1 = new Boolean("TRUE");
		Boolean b2 = new Boolean("true");
		if (b1 == b2)
		result = 1;
		if (b1.equals(b2))
		result = result + 10;
			2	CO3	K2

		System.out.println("result = " + result); } }			
7.		Discuss the roles of breakpoints in debugging.	2	CO3	K2
8.		Differentiate between syntax error and logic error.	2	CO3	K2
9.		Infer the purpose of the synchronized keyword.	2	CO3	K2
10.		Clarify what a Race condition is.	2	CO3	K2
11.		Contrast the start() and run() methods in threads.	2	CO3	K2
12.		Categorize the states of a thread life cycle.	2	CO3	K2
13.		Outline the operations of Java Collection API.	2	CO3	K2
14.		Differentiate between List and Set.	2	CO3	K2
15.		Differentiate between sleep() and wait().	2	CO3	K2

		Five Marks Questions.	**Marks**	**Course Outcome**	**Level**
1.		Develop a Java application to demonstrate multithreading by creating three separate threads. Each thread should concurrently read input from a file and perform specific tasks: counting vowels, counting consonants, and calculating the total number of characters.	5	CO3	K3
2.		Demonstrate synchronized methods and synchronized blocks with a suitable program.	5	CO3	K3
3.		Implement the Java program to create two threads: one for printing even numbers and another for printing odd numbers up to 10.	5	CO3	K3
4.		Implement a program to simulate inter-thread communication using wait() and notify(). Demonstrate proper synchronization.	5	CO3	K3
5.		Organize a Java application using Streams to sort a list of custom objects and group them by a specific property.	5	CO3	K3
6.		Implement a generic method to swap two elements in an array and illustrate its usage with examples.	5	CO3	K3
7.		Develop a JavaFX program to animate a bouncing ball and demonstrate the use of Timeline and KeyFrame.	5	CO3	K3
8.		Construct a program to simulate a producer-consumer scenario using threads and demonstrate inter-thread coordination.	5	CO3	K3
9.		Write a program to implement inter-thread communication using wait() and notify().	5	CO3	K3
10.		Explain the Java Collection Framework. Compare ArrayList, LinkedList, and Vector with respect to insertion, deletion, synchronization, and performance.	5	CO3	K3

Unit – IV

Unit 4 Data Base Connectivity

Unit Contents: Establish Connection to Data Base– Performing Create, Read, Update, Delete (CRUD) operations on Database – Insert, Delete – Update – Select - Join – Commit and Rollback – Debugging the JDBC Code.

S.No		**Two Marks Questions.**	**Marks**	**Course Outcome**	**Level**
1.		Distinguish between Statement and PreparedStatement.	2	CO4	K2
2.		Infer the purpose of the DriverManager class in JDBC.	2	CO4	K2
3.		Summarize the flow of a Hibernate-based web request from controller to database.	2	CO4	K2
4.		Restate the purpose of the SessionFactory in Hibernate using your own words.	2	CO4	K2
5.		Discuss any two types of thread synchronization used in Java.	2	CO4	K2
6.		Differentiate between commit() and rollback() in database transactions.	2	CO4	K2
7.		Illustrate how session management is handled in Servlets.	2	CO4	K2
8.		Discuss the need for ORM in Java applications.	2	CO4	K2
9.		Describe the responsibilities of Hibernate Session Factory in an application.	2	CO4	K2
10.		Describe the use of Hibernate configuration file (hibernate.cfg.xml).	2	CO4	K2
11.		Describe, with a small example, how Hibernate maps Java class fields to database columns?	2	CO4	K2
12.		Illustrate the purpose of HQL in Hibernate.	2	CO4	K2
13.		Describe the role of annotations in Hibernate.	2	CO4	K2
14.		Differentiate between first level and second level cache in terms of scope, default behavior, and configuration need.	2	CO4	K2
15.		Summarize the advantages of integrating Spring with Hibernate	2	CO4	K2

		Five Marks Questions.	**Marks**	**Course Outcome**	**Level**
1.		Implement the JDBC program to retrieve records using the SELECT statement.	5	CO4	K3
2.		Illustrate COMMIT and ROLLBACK operations using JDBC code.	5	CO4	K3

3.		Demonstrate the use of SQL commands such as INSERT, UPDATE, DELETE, and SELECT in a Java program using JDBC.	5	CO4	K3
4.		Demonstrate how to write and execute a Hibernate HQL query for retrieving all employee records.	5	CO4	K3
5.		Illustrate the steps involved in integrating Hibernate with Spring using an example configuration.	5	CO4	K3
6.		Develop a JDBC program to insert and view student details from a database.	5	CO4	K3
7.		Construct a simple JSP page that interacts with a servlet to display database results using JDBC.	5	CO4	K3
8.		Prepare a Hibernate configuration and mapping setup to store employee information in a database.	5	CO4	K3
9.		Demonstrate creating and executing HQL queries (SELECT, UPDATE, DELETE) with explanation.	5	CO4	K3
10.		Illustrate how to integrate Spring and Hibernate in a simple web application with proper configuration steps.	5	CO4	K3

Unit – V

Unit 5 Testing and Java 8 features

Unit Contents: Software Testing Life Cycle - JUnit - Introduction to Junit Annotations –- Configuring unit tests in Integrated Development Environments - Writing and executing unit tests- Java 8 features – Lambda expressions – Functional interfaces – Method references – Streams API – Date and Time API (java.time package) – Collectors API.

		Two Marks Questions.	**Marks**	**Course Outcome**	**Level**
1.		Illustrate the use of assertions in JUnit testing.	2	CO5	K2
2.		Summarize the functional interfaces required for lambda expressions.	2	CO5	K2
3.		Illustrate the phases involved in the Software Testing Life Cycle (STLC).	2	CO5	K2
4.		Compare traditional iteration and Streams API for data processing.	2	CO5	K2
5.		Compare unit testing and integration testing with respect to STLC.	2	CO5	K2
6.		Extrapolate the Lambda expressions in Java8.	2	CO5	K2
7.		Differentiate the save () and persist () methods in Hibernate.	2	CO5	K2
8.		Illustrate any two checks to perform when Hibernate entities are not detected in a Spring application.	2	CO5	K2
9.		Summarize the significance of loose coupling in enterprise-level applications.	2	CO5	K2
10.		Represent the role of SessionFactory in a Spring–Hibernate integrated environment.	2	CO5	K2
11.		Differentiate Spring MVC from Spring Web.	2	CO5	K2

12.		Illustrate how the @Controller annotation works in Spring MVC.	2	CO5	K2
13.		Summarize the workflow of Spring Boot auto-configuration.	2	CO5	K2
14.		Compare Spring JDBC with Hibernate based on data-access mechanisms.	2	CO5	K2
15.		Outline the steps involved in connecting a database using Spring JDBC.	2	CO5	K2

		Five Marks Questions.	**Marks**	**Course Outcome**	**Level**
1.		Utilize the java.time API to perform date and time calculations in a Java application.	5	CO5	K3
2.		Build a simple Java8 program to process a collection of objects using (i) Lambda expressions (ii) Method references (iii)Streams API Demonstrate their use with a suitable example.	5	CO5	K3
3.		Implement a Java program using Streams API to filter, map, and collect data from a list.	5	CO5	K3
4.		Infer the use of JUnit annotations by developing test cases for a simple Java class.	5	CO5	K3
5.		Illustrate the assertions used to validate program correctness in JUnit with a suitable scenario.	5	CO5	K3
6.		Develop a Java program using Streams API and Date–Time API (java.time package) to process and display data.	5	CO5	K3
7.		Illustrate the steps required to configure Hibernate within a Spring application and demonstrate how Spring manages the Hibernate SessionFactory	5	CO5	K3
8.		Build a Spring MVC application and demonstrate the request handling flow from Controller to View.	5	CO5	K3
9.		Apply Spring JDBC to connect a relational database and illustrate the process of executing CRUD operations.	5	CO5	K3
10.		Construct a RESTful Spring Web application and demonstrate how to use @RestController for handling HTTP requests.	5	CO5	K3